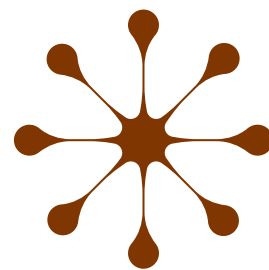




UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
Universidad del Perú, Decana de América



PROYECTO FINAL DE CIRCUITOS DIGITALES

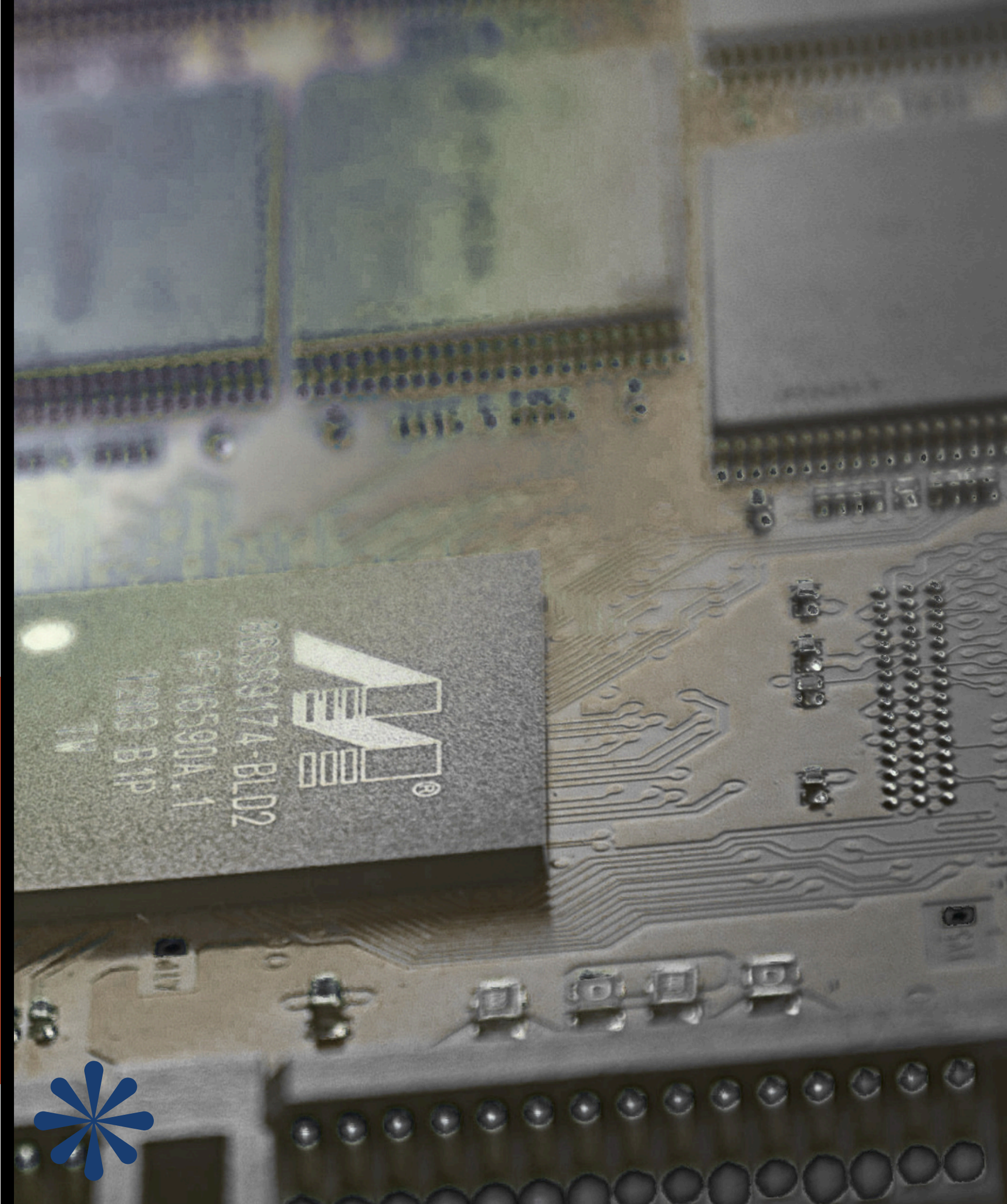
Integrantes:

Gomez Prada, Alonso Abraham

Silva Centeno, Bryan Rodolfo

Huiman Vargas, Juan Jesus

Zamora de la Cruz, Pablo Alonso

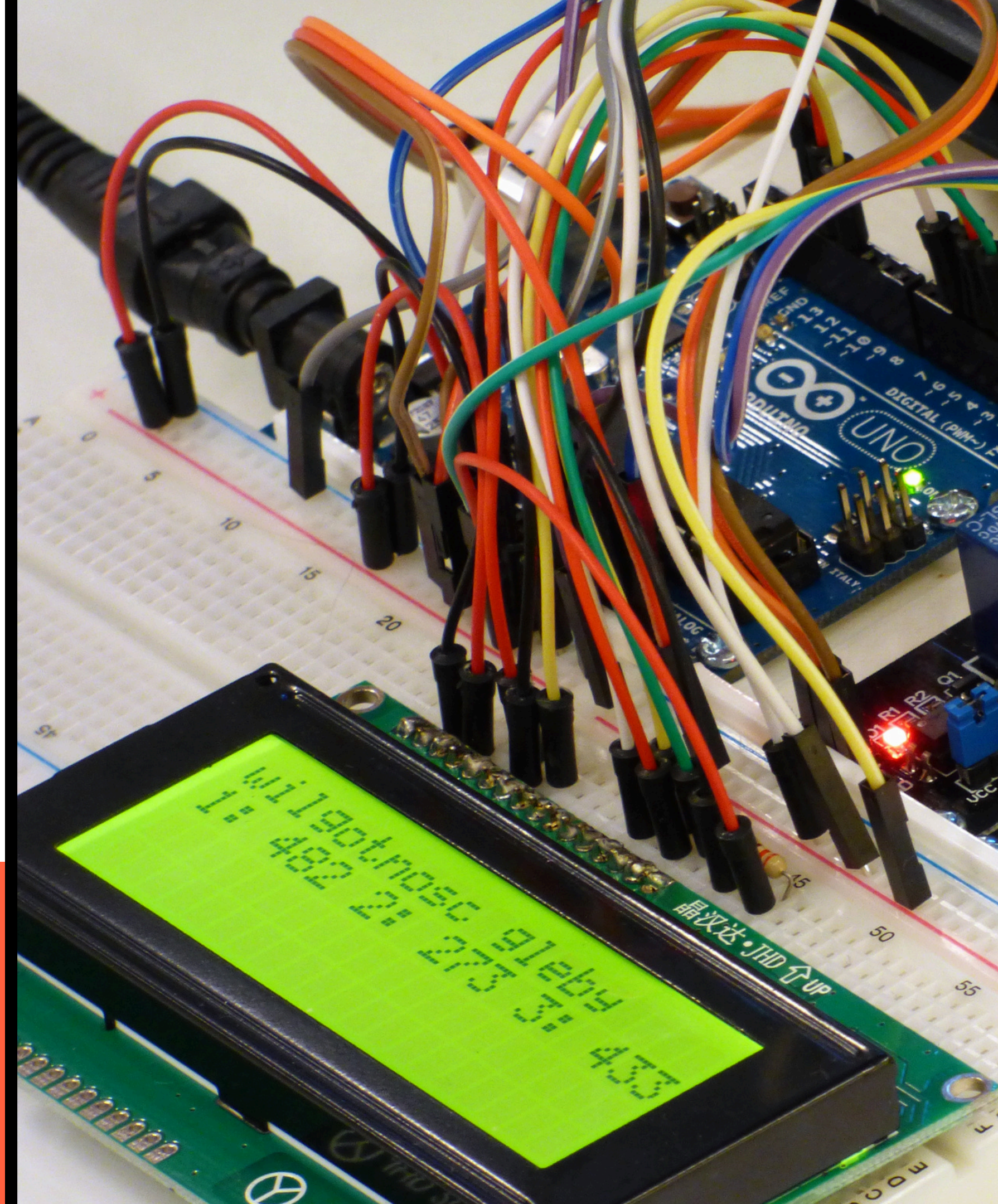


DESCRIPCIÓN DEL SISTEMA

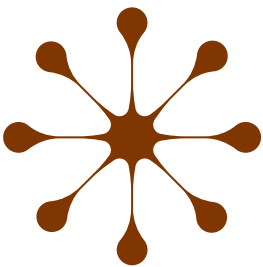
El emulador AT en Arduino Uno R3 procesa y transmite comandos AT de forma ordenada. Todo el flujo de trabajo y el control del sistema están gestionados de manera estricta por una Máquina de Estados Finita.

Para la interacción, el circuito utiliza un teclado matricial, una pantalla LCD 16×2 con interfaz I2C para mostrar el estado del sistema y un módulo GSM/GPRS SIM800L que permite realizar llamadas de voz. El monitoreo del estado operativo se complementa mediante indicadores LED.

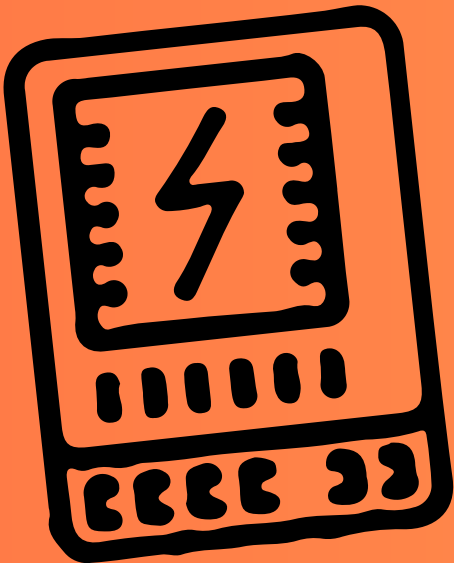
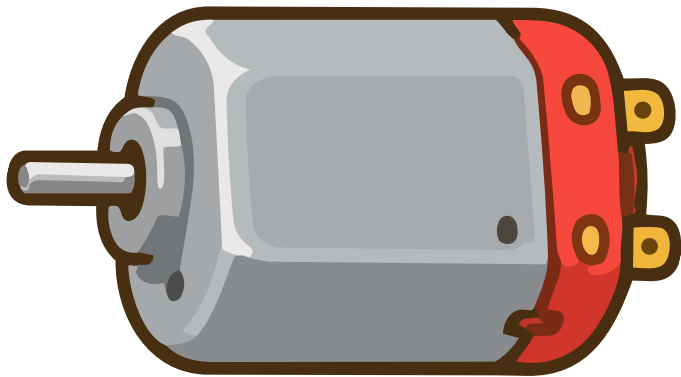
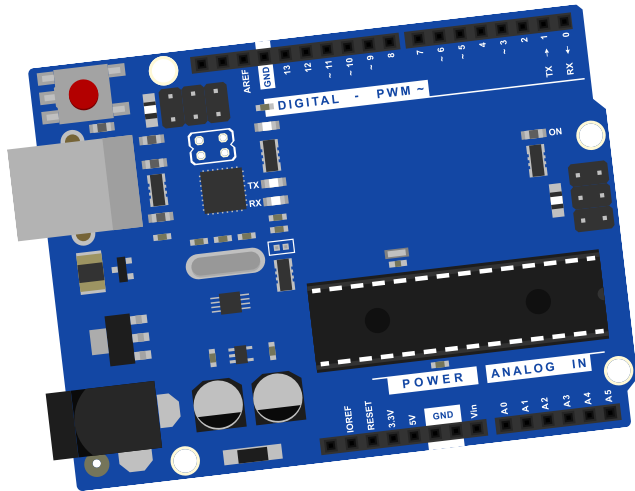
- ✓ Arquitectura modular y escalable.
- ✓ Bajo consumo de recursos del Arduino.
- ✓ Interfaz funcional mediante teclado y LCD.
- ✓ Integración con comunicación celular real.

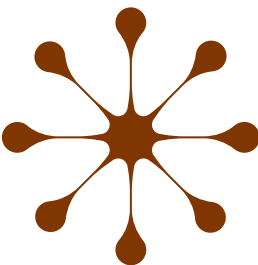


Componente	Cantidad	Función dentro del sistema
Arduino Uno R3	1	Unidad de procesamiento principal y ejecución de la Máquina de Estados Finita (FSM).
Módulo GSM/GPRS SIM800L	1	Comunicación celular para el envío de SMS mediante comandos AT.
Tarjeta SIM	1	Acceso a la red GSM para la comunicación celular.
Fuente DC 9–12 V	1	Alimenta el sistema y el regulador LM2596.
Regulador de voltaje LM2596 (Step-Down DC-DC)	1	Reduce la tensión de alimentación a 4.0 V aproximadamente para suministrar energía estable al módulo SIM800L, soportando los picos de corriente durante la transmisión.
Teclado matricial 4×4	1	Entrada de datos y selección de opciones del sistema.

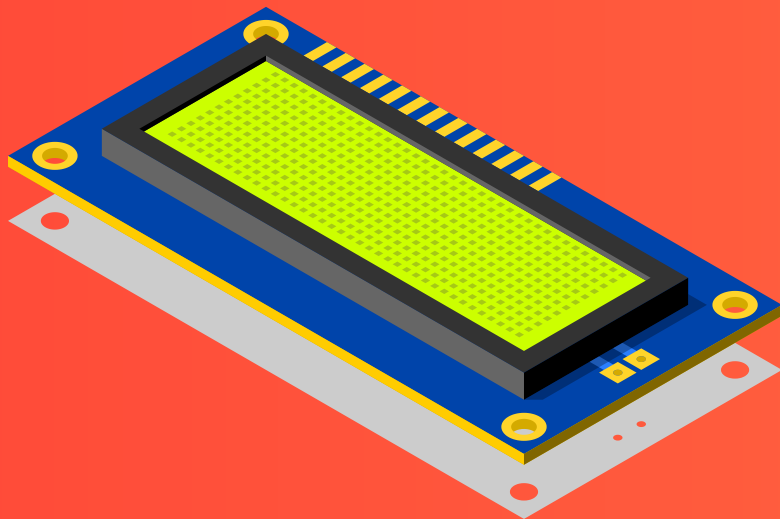
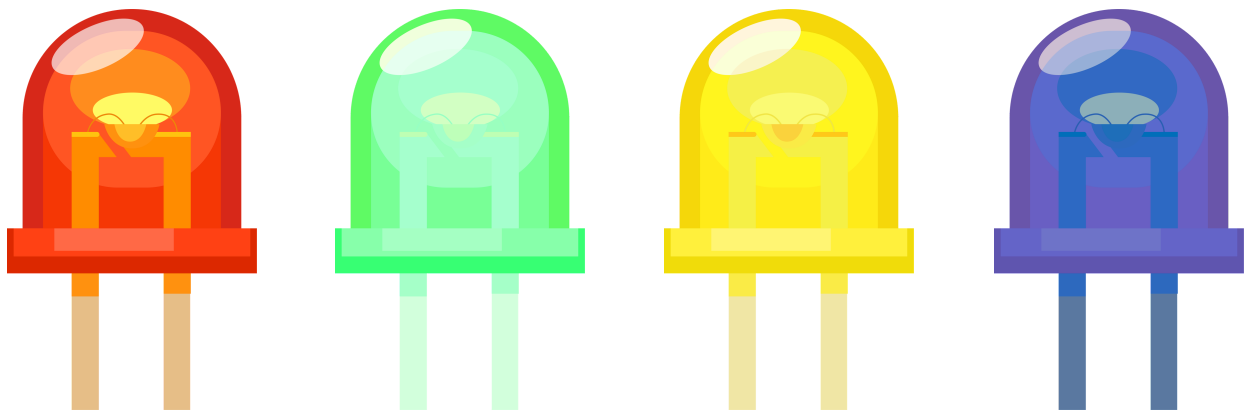


LISTA DE COMPONENTES





LISTA DE COMPONENTES



Componente	Cantidad	Función dentro del sistema
Pantalla LCD 16×2 con interfaz I2C	1	Muestra el menú principal, estados del sistema, mensajes de error y confirmaciones al usuario.
LED rojo	1	Indica la recepción de un nuevo mensaje SMS. Parpadea hasta que el usuario lo lea.
LED amarillo	1	Indica el modo de operación del sistema
LED verde	1	Confirma el envío exitoso del SMS.
LED azul	1	Indica el estado de conexión con la red GSM
Resistencias 220 Ω	4	Limitan la corriente de los LEDs.
Cables Dupont	20 - 30	Interconexión entre módulos.

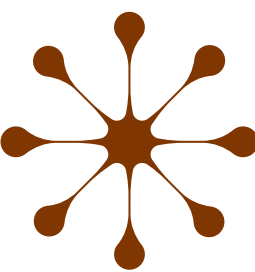
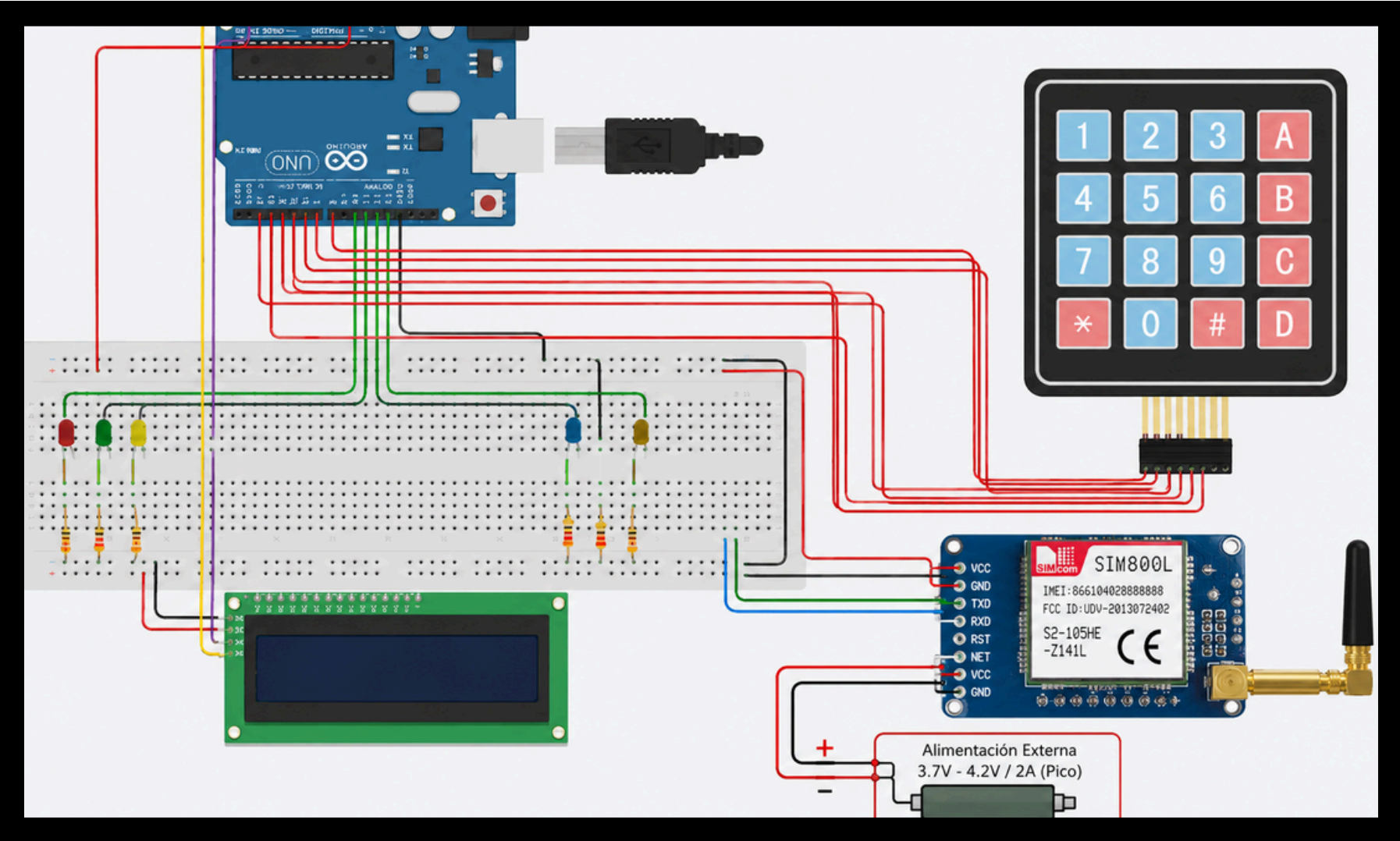


DIAGRAMA DE CONEXIONES



Componente Físico	Pin del componente	Pin en Arduino Uno	Función / Tipo de Señal
Pantalla LCD I2C	VCC	5 V	Alimentación (5 V DC)
	GND	GND	Referencia de Tierra
	SCL	A5	Reloj del Bus I2C (Serial Clock)
	SDA	A4	Datos del Bus I2C (Serial Data)

- Comunicación mediante protocolo I²C.
- Utiliza únicamente los pines A4 (SDA) y A5 (SCL).
- Muestra comandos, estados y mensajes de error.
- Reduce el uso de pines respecto a una pantalla LCD convencional.
- Muestra información del sistema en tiempo real.

Componente Físico	Pin del componente	Pin en Arduino Uno	Función / Tipo de Señal
Teclado Matricial 4x4	Fila 1	4	Salida Digital (Barrido FSM)
	Fila 2	5	Salida Digital (Barrido FSM)
	Fila 3	6	Salida Digital (Barrido FSM)
	Fila 4	7	Salida Digital (Barrido FSM)
	Columna 1	8	Entrada Digital (Internal Pull-Up)
	Columna 2	9	Entrada Digital (Internal Pull-Up)
	Columna 3	10	Entrada Digital (Internal Pull-Up)
	Columna 4	11	Entrada Digital (Internal Pull-Up)

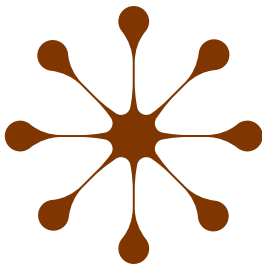
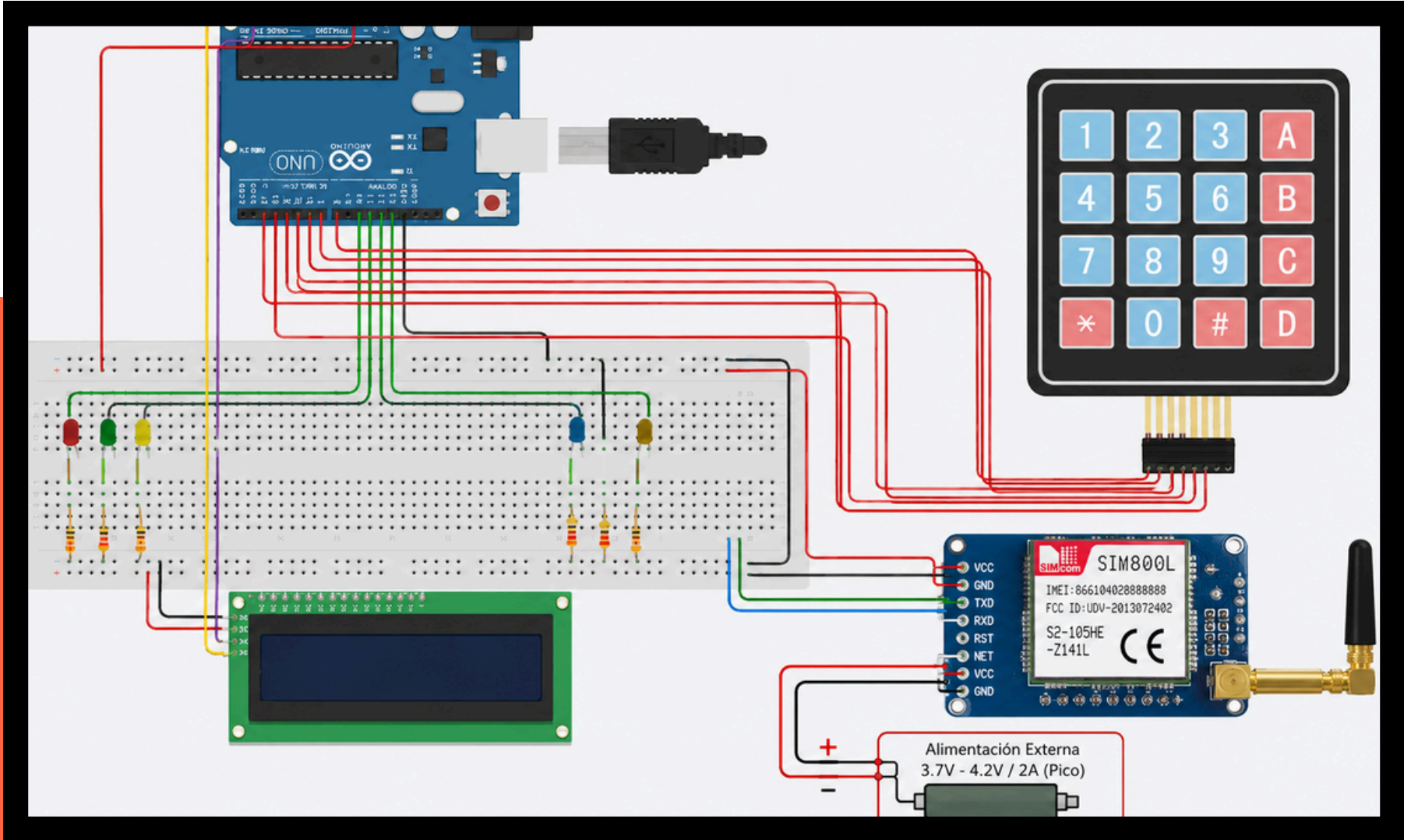


DIAGRAMA DE CONEXIONES



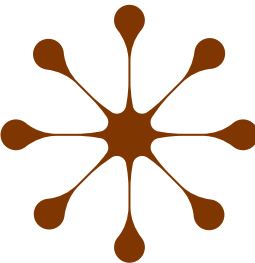
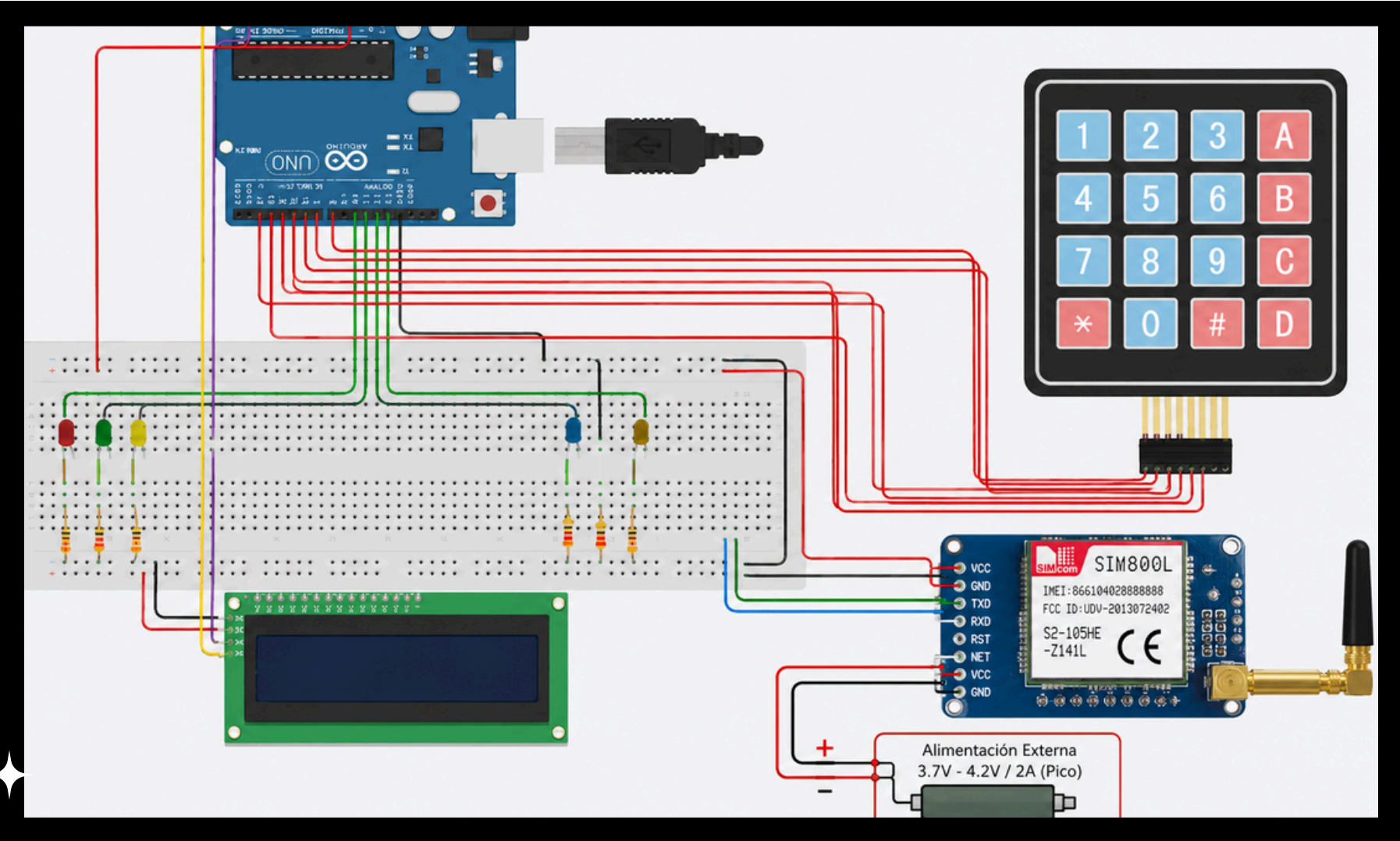


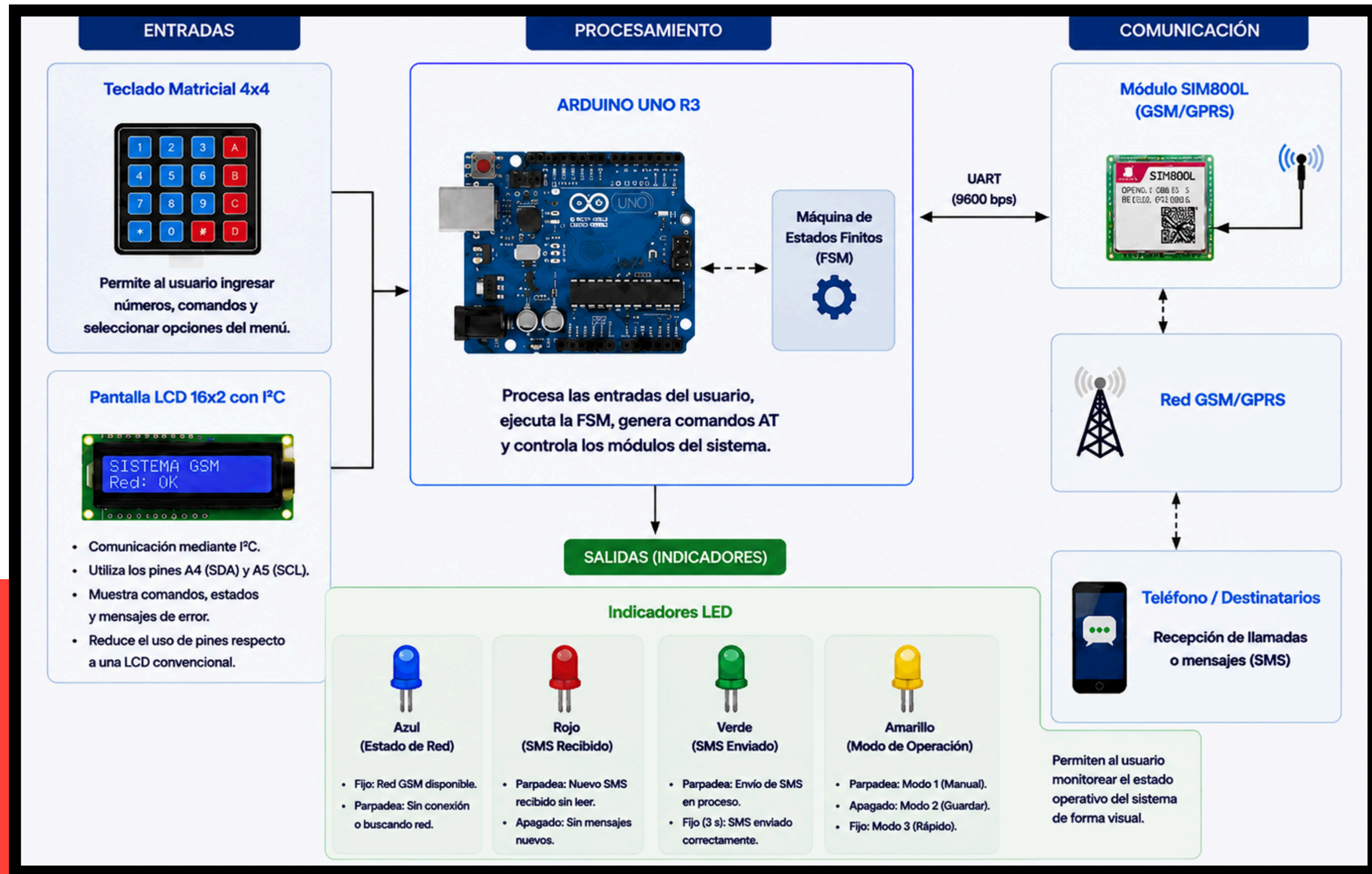
DIAGRAMA DE CONEXIONES

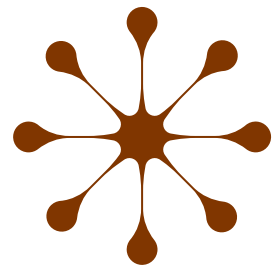
SIM800L	Arduino
VCC	Fuente 4V externa
GND	GND
TX	RX (3)
RX	TX (2)



- SIM800L GSM/GPRS: permite realizar llamadas y enviar comandos AT mediante comunicación UART.
- Red GSM: proporciona conectividad celular para la transmisión de voz y datos.
- UART 9600 bps: canal de comunicación entre el Arduino y el módulo SIM800L.

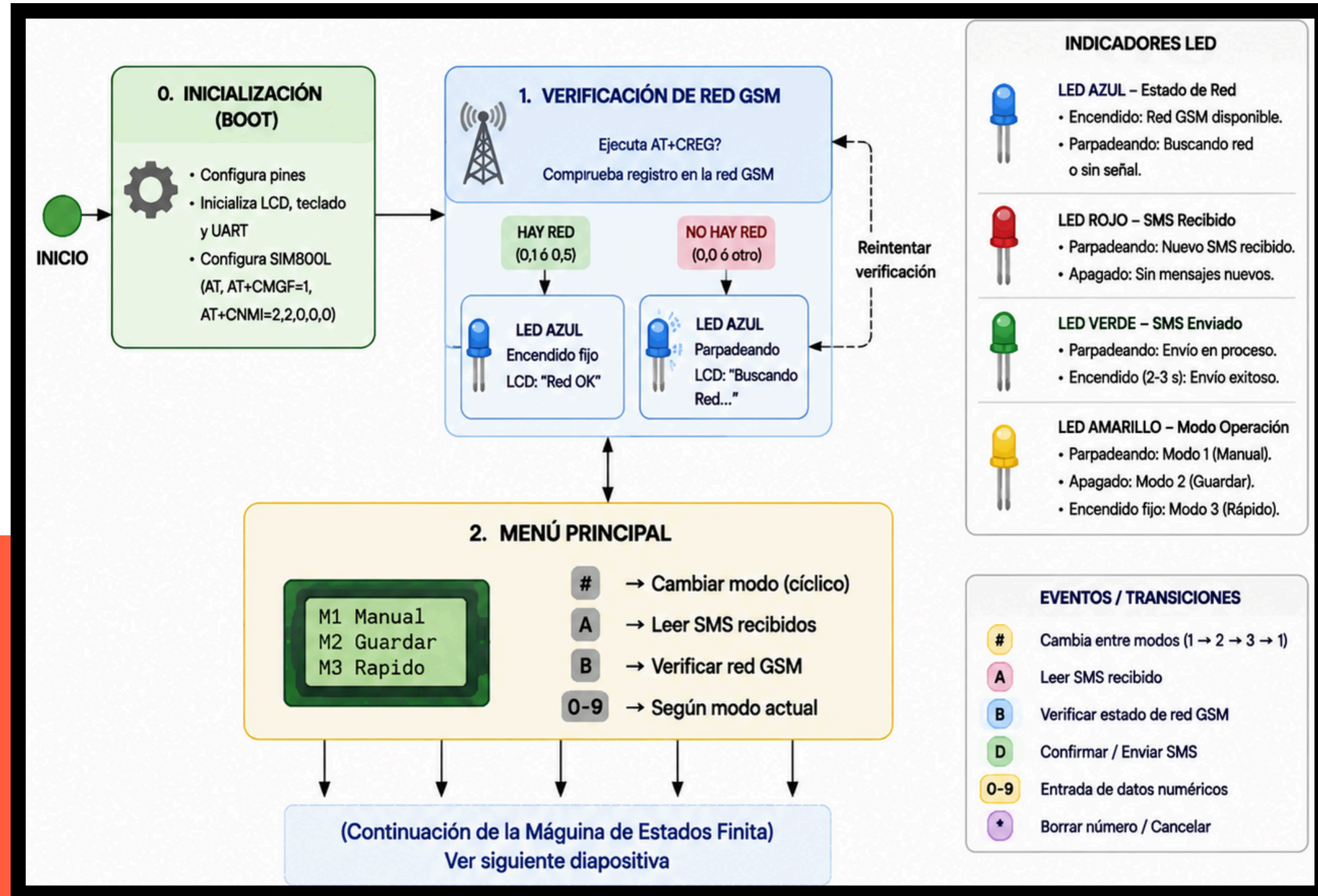
ARQUITECTURA DEL SISTEMA

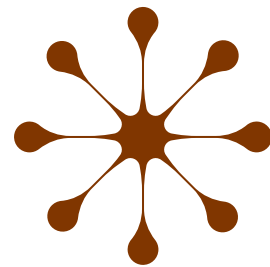




MAQUINA DE ESTADOS FINITOS

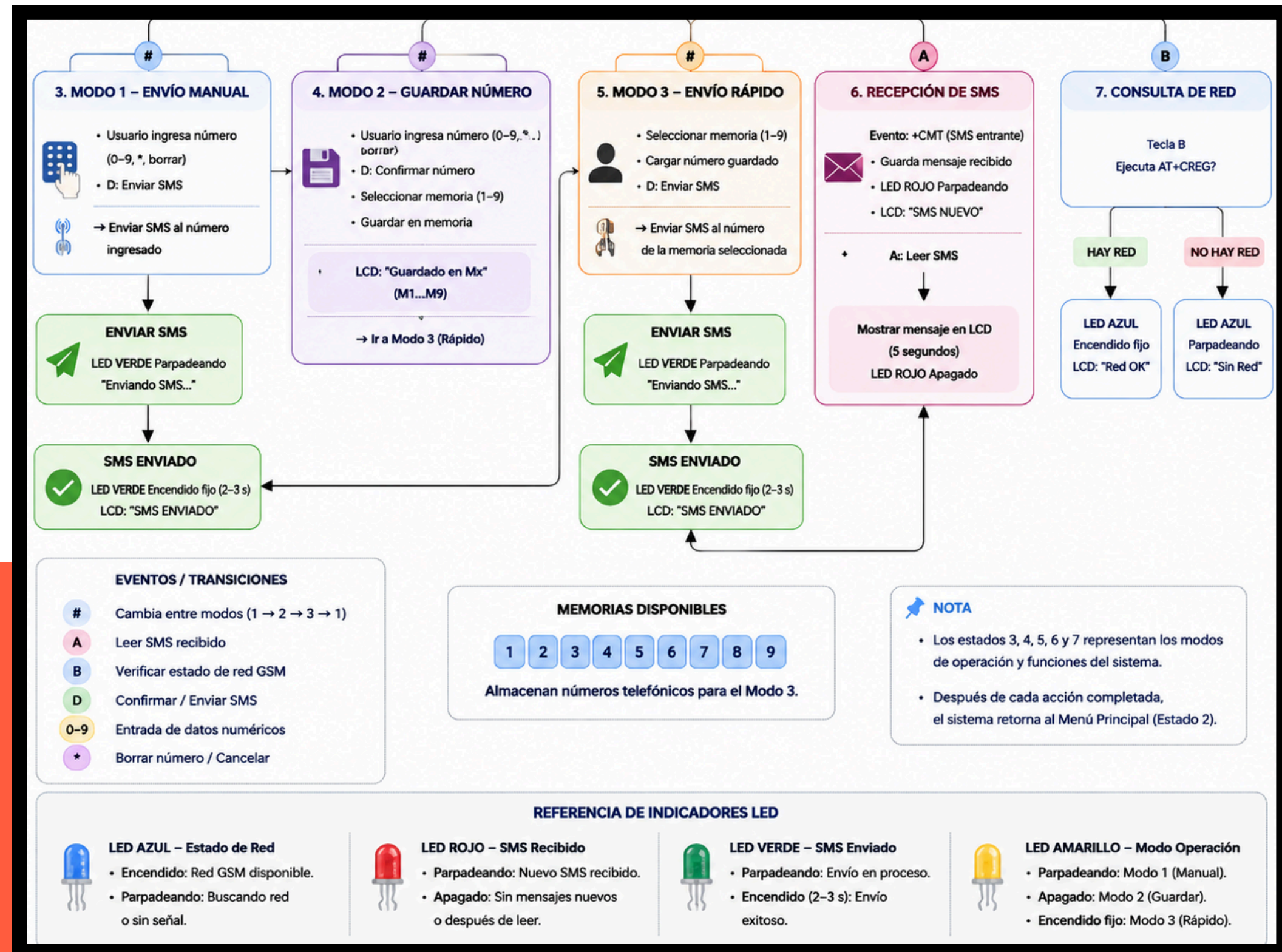
La Máquina de Estados Finitos (FSM) organiza el funcionamiento del sistema mediante estados bien definidos y transiciones controladas. Cada acción del usuario o respuesta del módulo SIM800L provoca un cambio de estado, garantizando un funcionamiento ordenado, confiable y libre de conflictos.

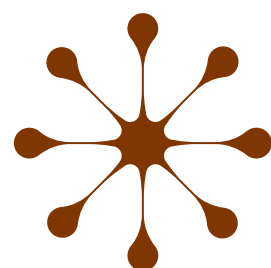




MAQUINA DE ESTADOS FINITOS

Esta sección muestra los estados de operación del sistema y las transiciones que permiten gestionar el envío y recepción de SMS, el almacenamiento de contactos, la consulta de la red GSM y el retorno automático al menú principal después de cada proceso.

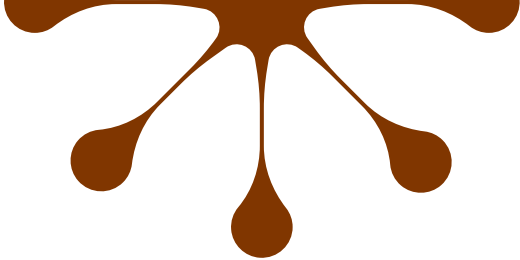




COMANDOS AT UTILIZADOS

Los comandos AT constituyen el conjunto de instrucciones utilizadas para controlar el módulo SIM800L desde el Arduino Uno mediante comunicación UART. A través de ellos es posible verificar el estado del módulo, consultar la red GSM y ejecutar funciones como el envío de SMS y llamadas.

Comando AT	Siglas	Descripción de la función
AT	Attention	Verifica la comunicación con el módulo SIM800L.
AT + IPR = 9600	Interface Port Rate	Configura la velocidad de comunicación serial del módulo a 9600 baudios.
AT + CREG?	Network Registration	Consulta si el módulo está registrado en la red GSM.
AT + CSQ	Signal Quality	Mide la intensidad de la señal GSM recibida.
AT + CMGF = 1	Configure Message Format	Configura el envío de SMS en modo texto.
AT + CMGS = "+51 (número)"	Command Message Send	Envía un mensaje SMS al número especificado.
AT + CNMI = 2,2,0,0,0	New Message Indications to Terminal Equipment	Configura el módulo para que los SMS recibidos sean enviados directamente al Arduino.



CÓDIGO EN ARDUINO

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <SoftwareSerial.h>

// =====
// CONFIGURACIÓN DE PINES
// =====
// Cambiamos la velocidad interna a 9600 para máxima estabilidad
SoftwareSerial sim800(2, 3);
LiquidCrystal_I2C lcd(0x27, 16, 2);

#define LED_AZUL A0
#define LED_ROJO A1
#define LED_VERDE A2
#define LED_AMARILLO A3

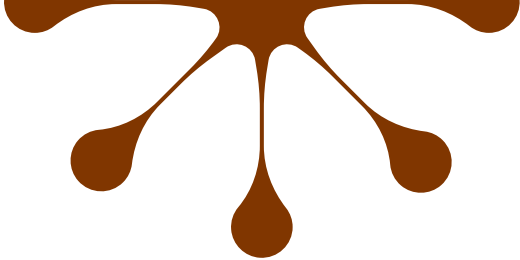
const byte FILAS = 4;
const byte COLUMNAS = 4;
char keys[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
```

```
};

byte pinesFilas[FILAS] = {4, 5, 6, 7};
byte pinesColumnas[COLUMNAS] = {8, 9, 10, 11};
Keypad teclado = Keypad(makeKeymap(keys), pinesFilas, pinesColumnas, FILAS, COLUMNAS);

// =====
// VARIABLES GLOBALES
// =====

string numeroCelular = "";
string ultimoMensaje = "";
string agenda[10];
bool hayMensajeNuevo = false;
bool redConectada = false;
```

CÓDIGO EN ARDUINO

```
int modoActual = 1;
int pasoModo2 = 0;
int memoriaTemp = -1;

unsigned long tiempoAnteriorBlink = 0;
bool estadoBlink = false;

// =====
// PROTOTIPOS DE FUNCIONES
// =====

void actualizarPantalla();
void enviarSMSEmergencia(String numero);
void verificarRed();
void procesarModo1(char tecla);
void procesarModo2(char tecla);
void procesarModo3(char tecla);
```

```
// =====
// SETUP
// =====

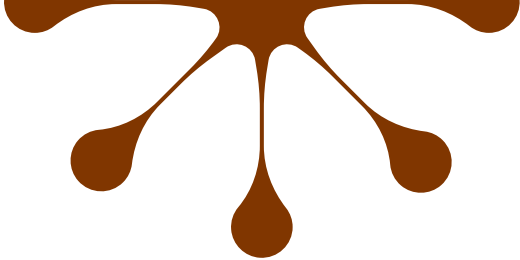
void setup() {
  Serial.begin(115200); // PC a 115200
  sim800.begin(9600);   // ¡SIM800L cambiado a 9600 para estabilidad!

  pinMode(LED_AZUL, OUTPUT);
  pinMode(LED_ROJO, OUTPUT);
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_AMARILLO, OUTPUT);

  lcd.init();
  lcd.backlight();

  Serial.println(F("\n====="));
  Serial.println(F("  INICIANDO CENTRAL DE ALARMAS  "));
  Serial.println(F("====="));
  Serial.println(F("[SISTEMA] Configurando modulo SIM800L..."));

  lcd.setCursor(0, 0);
  lcd.print(F("Iniciando red..."));
```

CÓDIGO EN ARDUINO

```
// Forzamos al módulo a sincronizarse a 9600 baudios
sim800.println(F("AT"));
delay(500);
sim800.println(F("AT+IPR=9600")); // Comando para fijar 9600 baudios perman
delay(500);
sim800.println(F("AT+CMGF=1"));
delay(500);
sim800.println(F("AT+CNMI=2,2,0,0,0"));
delay(1000);

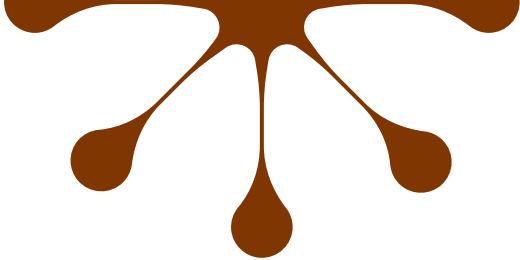
verificarRed();
Serial.println(F("\n[SISTEMA] Listo. Presione '#' para cambiar de modo.));
actualizarPantalla();
```

```
// =====
// LOOP PRINCIPAL
// =====
void loop() {
    // 1. Control de LEDs
    unsigned long tiempoActual = millis();
    if (tiempoActual - tiempoAnteriorBlink >= 500) {
        tiempoAnteriorBlink = tiempoActual;
        estadoBlink = !estadoBlink;

        if (modoActual == 1) digitalWrite(LED_AMARILLO, estadoBlink);
        else if (modoActual == 2) digitalWrite(LED_AMARILLO, LOW);
        else if (modoActual == 3) digitalWrite(LED_AMARILLO, HIGH);

        if (hayMensajeNuevo) digitalWrite(LED_ROJO, estadoBlink);
        else digitalWrite(LED_ROJO, LOW);

        if (redConectada) digitalWrite(LED_AZUL, HIGH);
        else digitalWrite(LED_AZUL, estadoBlink);
    }
}
```



CÓDIGO EN ARDUINO

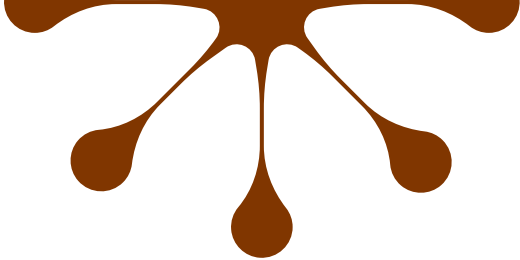
```
// 2. LECTURA DEL TECLADO
char tecla = teclado.getKey();

if (tecla) {
  if (tecla == '#') {
    modoActual++;
    if (modoActual > 3) modoActual = 1;
    numeroCelular = "";
    pasoModo2 = 0;

    Serial.println(F("\n-----"));
    Serial.print(F("[TECLADO] Cambio de Modo -> MODO "));
    Serial.println(modoActual);
    Serial.println(F("-----"));

    actualizarPantalla();
  }
  else if (tecla == 'A') {
    Serial.println(F("\n[TECLADO] Boton 'A' presionado (Bandeja)"));
    if (hayMensajeNuevo) {
      Serial.print(F("[MENSAJE] Leyendo: "));
      Serial.println(ultimoMensaje);
      lcd.clear();
      lcd.setCursor(0, 0); lcd.print(F("SMS RECIBIDO:"));
      lcd.setCursor(0, 1); lcd.print(ultimoMensaje.substring(0, 16));
    }
  }
}
```

```
    delay(5000);
    hayMensajeNuevo = false;
    actualizarPantalla();
  } else {
    Serial.println(F("[MENSAJE] Bandeja vacia."));
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print(F("Bandeja vacia"));
    delay(2000); actualizarPantalla();
  }
}
else if (tecla == 'B') {
  Serial.println(F("\n[TECLADO] Boton 'B' presionado (Senal)"));
  verificarRed();
}
else {
  if (modoActual == 1) procesarModo1(tecla);
  else if (modoActual == 2) procesarModo2(tecla);
  else if (modoActual == 3) procesarModo3(tecla);
}
}
```

CÓDIGO EN ARDUINO

```
// 3. LECTURA DE SMS EN SEGUNDO PLANO
if (sim800.available()) {
    String respuestaSIM = sim800.readString();

    // Imprime TODO lo que responda el módulo en el monitor para ver qué pasa en vivo
    Serial.print(F("[SIM800 RAW]: "));
    Serial.println(respuestaSIM);

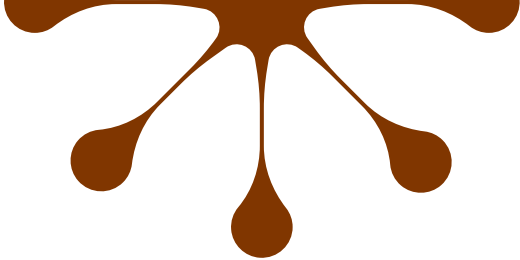
    if (respuestaSIM.indexOf(F("+CMT:")) != -1) {
        int saltoDeLinea = respuestaSIM.indexOf('\n', respuestaSIM.indexOf(F("+CMT:")));
        if (saltoDeLinea != -1) {
            ultimoMensaje = respuestaSIM.substring(saltoDeLinea + 1);
            ultimoMensaje.trim();
            hayMensajeNuevo = true;
            Serial.println(F("\n[ALERTA] ¡NUEVO SMS DETECTADO! Presione 'A' para leerlo en el LCD."));
            actualizarPantalla();
        }
    }
}
}
```



CÓDIGO EN ARDUINO

```
// =====  
// FUNCIONES DE LOS MODOS  
// =====  
void procesarModo1(char tecla) {  
    if (tecla >= '0' && tecla <= '9') {  
        if (numeroCelular.length() < 9) { numeroCelular += tecla; actualizarPantalla(); }  
    } else if (tecla == '*') {  
        numeroCelular = ""; actualizarPantalla();  
    } else if (tecla == 'D') {  
        if (numeroCelular.length() == 9) {  
            enviarSMSEmergencia(numeroCelular);  
            numeroCelular = ""; actualizarPantalla();  
        }  
    }  
}
```

```
void procesarModo2(char tecla) {  
    if (pasoModo2 == 0) {  
        if (tecla >= '0' && tecla <= '9') {  
            if (numeroCelular.length() < 9) { numeroCelular += tecla; actualizarPantalla(); }  
        } else if (tecla == '*') {  
            numeroCelular = ""; actualizarPantalla();  
        } else if (tecla == 'D' && numeroCelular.length() == 9) {  
            pasoModo2 = 1;  
            Serial.println(F("[INFO] Elija ranura (1-9) para guardar."));  
            actualizarPantalla();  
        }  
    }  
    else if (pasoModo2 == 1) {  
        if (tecla >= '1' && tecla <= '9') {  
            int espacio = tecla - '0';  
            agenda[espacio] = numeroCelular;  
  
            Serial.print(F("[GUARDADO] Numero ")); Serial.print(numeroCelular);  
            Serial.print(F(" en ranura ")); Serial.println(espacio);  
  
            lcd.clear();  
            lcd.setCursor(0,0); lcd.print(F("Guardado en:"));  
            lcd.setCursor(0,1); lcd.print(F("Espacio ")); lcd.print(espacio);  
            delay(2000);  
            numeroCelular = "";
```

CÓDIGO EN ARDUINO

```
numeroCelular = "";
pasoModo2 = 0;
modoActual = 3;
actualizarPantalla();
} else if (tecla == '*') {
    pasoModo2 = 0; actualizarPantalla();
}
}

void procesarModo3(char tecla) {
    if (tecla >= '1' && tecla <= '9') {
        int espacio = tecla - '0';
        if (agenda[espacio].length() == 9) {
            memoriaTemp = espacio;
            numeroCelular = agenda[espacio];
            Serial.print(F("[MEMORIA] Ranura ")); Serial.print(espacio); Serial.print(F(" cargada: ")); Serial.println(numeroCelular);
            actualizarPantalla();
        } else {
            lcd.clear(); lcd.setCursor(0,0); lcd.print(F("Espacio vacio")); delay(1500); actualizarPantalla();
        }
    } else if (tecla == '*') {
        numeroCelular = ""; memoriaTemp = -1; actualizarPantalla();
    } else if (tecla == 'D') {
        if (numeroCelular.length() == 9) {
```

CÓDIGO EN ARDUINO

```
if (numeroCelular.length() == 9) {
    enviarSMSEmergencia(numeroCelular);
    numeroCelular = ""; memoriaTemp = -1;
    actualizarPantalla();
}

/ =====
/ FUNCIONES AUXILIARES
/ =====

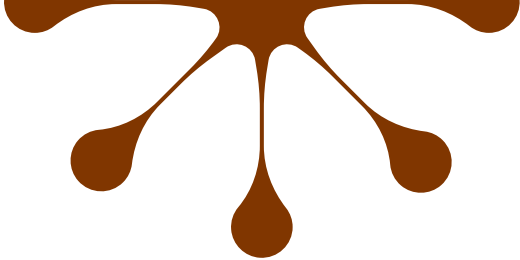
void actualizarPantalla() {
    lcd.clear();
    if (hayMensajeNuevo) {
        lcd.setCursor(0, 0); lcd.print(F("SMS NUEVO! (A)"));
    } else {
        if (modoActual == 1) { lcd.setCursor(0, 0); lcd.print(F("M1:Escriba Num:")); }
        else if (modoActual == 2) {
            if (pasoModo2 == 0) { lcd.setCursor(0, 0); lcd.print(F("M2:Num a guardar")); }
            else { lcd.setCursor(0, 0); lcd.print(F("M2:Elija slot 1-9")); }
        } else if (modoActual == 3) { lcd.setCursor(0, 0); lcd.print(F("M3:SOS Rapido")); }
    }
    lcd.setCursor(0, 1);
    lcd.print(numeroCelular);
}
```

```
void enviarSMSEmergencia(String numero) {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print(F("Enviando SMS..."));

    sim800.println(F("AT+CMGF=1")); delay(300);
    sim800.print(F("AT+CMGS=\""+51")); sim800.print(numero); sim800.println(F("\n")); delay(1000);
    sim800.print(F("ALERTA S.O.S: Mensaje de emergencia.")); delay(500);
    sim800.write(26);

    for(int i=0; i<6; i++){
        digitalWrite(LED_VERDE, HIGH); delay(400);
        digitalWrite(LED_VERDE, LOW); delay(400);
    }

    digitalWrite(LED_VERDE, HIGH);
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print(F("SMS ENVIADO"));
    delay(2000);
    digitalWrite(LED_VERDE, LOW);
}
```

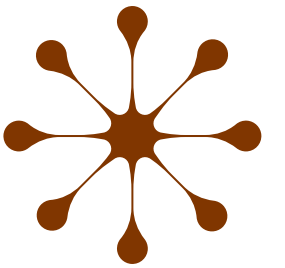



CÓDIGO EN ARDUINO

```
void verificarRed() {  
  sim800.println(F("AT+CREG?"));  
  delay(500);  
  if (sim800.available()) {  
    String res = sim800.readString();  
    if (res.indexOf(F("0,1")) != -1 || res.indexOf(F("0,5")) != -1) {  
      redConectada = true;  
      Serial.println(F("[RED] OK - Conectado."));  
    } else {  
      redConectada = false;  
      Serial.println(F("[RED] ERROR - Sin senal."));  
    }  
  }  
}
```



REPORTE DE RECURSOS



Uso de Memoria Flash

Parámetro	Valor
Mem. Flash Utilizada	12736 Bytes
Mem. Flash Libre	19520 Bytes
Mem. Flash Total	32256 Bytes
Porcentaje Utilizado	39%

Uso de Memoria Ram

Parámetro	Valor
Mem. RAM Utilizada	791 Bytes
Mem. RAM Libre	1257 Bytes
Mem. RAM Total	2048 Bytes
Porcentaje Utilizado	38%

El programa utiliza 12736 bytes de memoria Flash (39%) y 791 bytes de memoria RAM (38%), manteniendo recursos disponibles para futuras ampliaciones y garantizando un funcionamiento estable y eficiente del sistema.

APLICACIONES

Sistema de Alerta Antirrobo por GSM

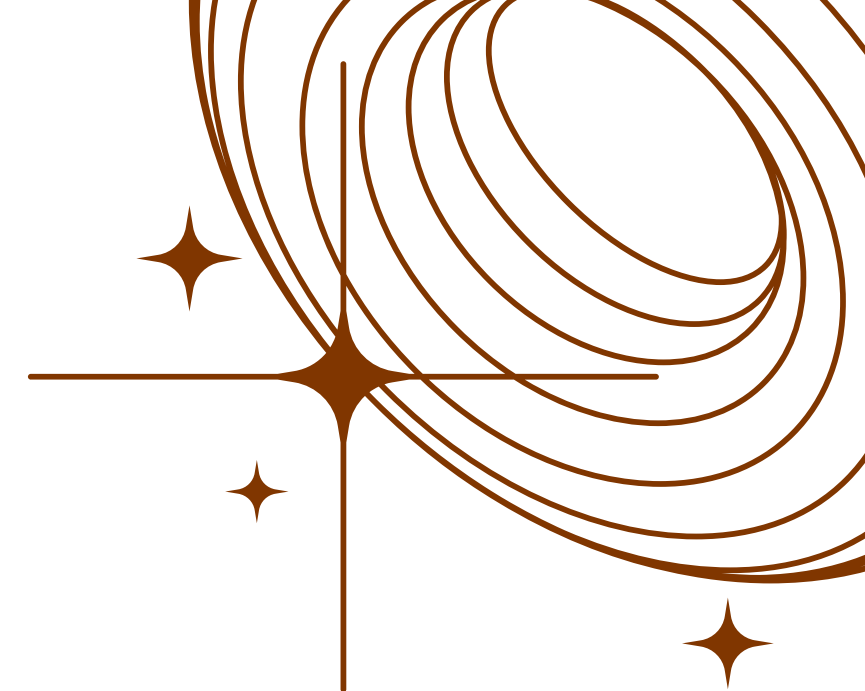
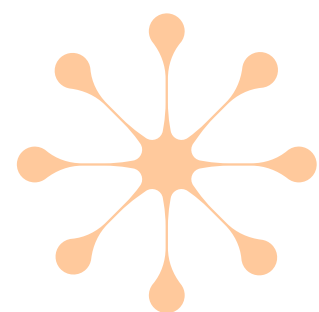
Cuando un sensor de movimiento o una alarma detecta un intento de robo, el Arduino ejecuta el envío automático de un SMS mediante el módulo SIM800L al propietario o al personal de seguridad. Además, el teclado matricial permite realizar acciones como consultar el estado del sistema o enviar alertas manuales.



Sistema de Bloqueo y Rastreo de Emergencia para Transporte de Carga

Cuando la central detecta una emergencia, el Arduino ejecuta el bloqueo del motor mediante un SMS recibido por el módulo SIM800L. Además, el teclado matricial permite al conductor ingresar el PIN de arranque o activar una alerta silenciosa, utilizando los indicadores LED para verificar visualmente si el sistema está bloqueado o activo.





¡Gracias!

